

# Homework 5: Breaking the Caesar Cipher with Grover's Algorithm

Quantum Computing Training 2026

## Background: The Caesar Cipher

The Caesar cipher is one of the oldest substitution ciphers. It is commonly associated with Julius Caesar, who is said to have used a fixed shift of the Roman alphabet to protect military messages. The modern version uses the 26-letter English alphabet. We encode

$$a = 0, \quad b = 1, \quad c = 2, \quad \dots, \quad z = 25.$$

A Caesar key is a shift

$$k \in \mathbb{Z}_{26}.$$

Encryption of one letter is

$$\text{Caesar}_k(p) = p + k \pmod{26},$$

and decryption is

$$p = c - k \pmod{26}.$$

Spaces and punctuation are usually left unchanged.

**Example.** Using the shift  $k = 3$ , the letter  $a$  becomes  $d$ ,  $b$  becomes  $e$ , and  $c$  becomes  $f$ , and so on... Thus

the cow jumped over the moon

encrypts to

wkh frz mxpshg ryhu wkh prrq.

This cipher is easy to break classically because there are only 26 possible shifts. In this homework, we use it as a toy model for understanding how Grover's algorithm searches over candidate keys.

## Problem 1: Caesar Key Recovery as Grover Search

For Problems 1–4, use a toy alphabet of size eight,

$$\mathcal{A}_8 = \{0, 1, 2, 3, 4, 5, 6, 7\},$$

with Caesar encryption

$$\text{Caesar}_k(p) = p + k \pmod{8}.$$

The key has eight possible values, so the key register has three qubits. Suppose you are given one known plaintext-ciphertext pair

$$p = 3, \quad c = 6,$$

where

$$c = \text{Caesar}_{k^*}(p)$$

for an unknown key  $k^* \in \mathbb{Z}_8$ .

1.a. Compute  $k^*$  by hand.

1.b. Define

$$f(k) = \begin{cases} 1, & \text{Caesar}_k(p) = c, \\ 0, & \text{Caesar}_k(p) \neq c. \end{cases}$$

Make a table of  $f(k)$  for  $k = 0, 1, \dots, 7$ .

1.c. Identify the search space size  $N$  and the number of marked items  $M$ . How many Grover iterations does suggest?

$$\text{Number of iterations} \approx \left\lceil \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rceil$$

1.d. Write the initial key state

$$|s\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^7 |k\rangle.$$

Which basis state should receive a minus sign from the Grover oracle?

## Problem 2: A Reversible Caesar Encryption Block

The oracle should not hardcode the answer  $k^*$ . It should check each candidate key using the public test

$$\text{Caesar}_k(p) = p + k \pmod{8} = c.$$

Use two registers:

$$|k\rangle |y\rangle,$$

where  $|k\rangle$  is a 3-qubit key register and  $|y\rangle$  is a 3-qubit work register.

2.a. The reversible encryption block should compute

$$U_E : |k\rangle |0\rangle \mapsto |k\rangle |3 + k \bmod 8\rangle.$$

Make a table of the work-register value  $y = k + 3 \bmod 8$  for all eight keys.

2.b. Implement this reversible block in Qiskit. You may use any correct reversible method, such as a small permutation/unitary for this toy problem or a hand-built 3-bit modular adder.

2.c. Test your implementation on all basis states  $|k\rangle |000\rangle$ . Verify that the key register is unchanged and the work register contains  $3 + k \bmod 8$ .

2.d. Explain why  $U_E$  is only a checker/encryption block, not yet a Grover phase oracle.

## Problem 3: Constructing the Phase Oracle

In Problem 2, you built a reversible Caesar encryption block

$$U_E : |k\rangle |0\rangle \mapsto |k\rangle |3 + k \bmod 8\rangle.$$

The first register is the candidate key register, and the second register is the work register. For our toy instance, the known plaintext-ciphertext pair is still

$$p = 3, \quad c = 6.$$

Thus the correct key is the unique  $k \in \mathbb{Z}_8$  satisfying

$$3 + k \equiv 6 \pmod{8}.$$

Grover's algorithm needs a phase oracle on the key register:

$$O_f |k\rangle = (-1)^{f(k)} |k\rangle,$$

where

$$f(k) = \begin{cases} 1, & 3 + k \equiv 6 \pmod{8}, \\ 0, & 3 + k \not\equiv 6 \pmod{8}. \end{cases}$$

So the correct key should receive a minus sign, and all incorrect keys should be unchanged.

**3.a.** From Problem 1, what is the correct key  $k^*$ ? Which three-qubit basis state  $|k^*\rangle$  should receive a minus sign?

**3.b.** After applying  $U_E$ , the work register contains

$$y = 3 + k \bmod 8.$$

To mark the correct key, we need to apply a minus sign exactly when

$$y = 6.$$

Make a table with columns  $k$ ,  $y = 3 + k \bmod 8$ , and whether  $y = 6$ .

**3.c.** We now construct a comparison phase on the work register. The goal is the operation

$$|y\rangle \mapsto \begin{cases} -|y\rangle, & y = 6, \\ |y\rangle, & y \neq 6. \end{cases}$$

Since  $6 = 110_2$ , explain which work-register basis state should receive the minus sign.

**3.d.** To implement the comparison phase, first transform the target state  $|110\rangle$  into  $|111\rangle$ , because a multi-controlled phase gate naturally fires when all control qubits are  $|1\rangle$ .

Which bit of 110 is 0? Which  $X$  gate should you apply to turn

$$|110\rangle$$

into

$$|111\rangle?$$

**3.e.** After that  $X$  gate, apply a three-qubit controlled phase that multiplies  $|111\rangle$  by  $-1$  and leaves all other work-register basis states unchanged. Then undo the  $X$  gate from part (d).

Explain why the net effect is a minus sign exactly on  $|110\rangle$ , i.e. exactly when  $y = 6$ .

**3.f.** Now combine the pieces. The full Caesar phase oracle is

$$O_f = U_E^\dagger (\text{phase flip on } y = 6) U_E.$$

Track the action on a basis state:

$$|k\rangle |000\rangle \xrightarrow{U_E} |k\rangle |3 + k \bmod 8\rangle \xrightarrow{\text{phase flip on } y=6} \text{_____} \xrightarrow{U_E^\dagger} \text{_____}$$

Fill in the blanks.

**3.g.** What is the final state of the work register after the full oracle? Why is it important that the work register returns to  $|000\rangle$ ?

**3.h.** Implement this phase oracle in Qiskit:

- (a) apply your  $U_E$  from Problem 2;
- (b) apply the comparison phase that flips the sign when the work register is  $|110\rangle$ ;
- (c) apply  $U_E^\dagger$ .

Test your oracle on all eight basis states  $|k\rangle |000\rangle$ . Verify that only the correct key branch receives a minus sign and that the work register returns to  $|000\rangle$ .

**3.i.** Finally, apply your oracle to the uniform key superposition

$$|s\rangle |000\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^7 |k\rangle |000\rangle.$$

Write the expected output state after the oracle. Which amplitude has changed sign?

## Problem 4: Full Grover Search in Qiskit

Now combine the Caesar phase oracle with the Grover diffuser.

**4.a.** Build the three-qubit diffuser on the key register:

$$D = H^{\otimes 3} (2|000\rangle\langle 000| - I) H^{\otimes 3}.$$

Implement the middle reflection using  $X$  gates and a multi-controlled phase flip.

**4.b.** Assemble one Grover iteration

$$G = DO_f.$$

Run the circuit for  $R = 0, 1, 2, 3$  Grover iterations. Plot the probability of measuring each key.

**4.c.** For each  $R$ , record the probability of measuring the correct key. Does the success probability keep increasing forever?

**4.d.** Repeat the experiment for two other plaintext-ciphertext pairs of your choice in  $\mathcal{A}_8$ . Verify that your oracle marks the correct key without hardcoding the key itself.

## Problem 5: Full 26-Letter Caesar Cipher

Now return to the ordinary Caesar cipher on the 26-letter alphabet.

**5.a.** How many qubits are needed to represent one letter? Explain why 4 qubits are not enough and 5 qubits are enough.

**5.b.** A 5-qubit register has 32 computational basis states, but only 26 correspond to letters. We will call

$$0, 1, \dots, 25$$

valid letter states and

$$26, 27, 28, 29, 30, 31$$

unused states. Choose and clearly state a convention for what your circuit does on unused states. Your convention must make the overall operation reversible.

**5.c.** Let the known plaintext-ciphertext pair be the first letters of the example above:

$$p = \mathfrak{t} = 19, \quad c = \mathfrak{w} = 22.$$

The correct key should be  $k^* = 3$ . Define a reversible block that computes

$$|k\rangle |0\rangle \mapsto |k\rangle |(p+k) \bmod 26\rangle$$

for valid keys  $k = 0, 1, \dots, 25$ . Specify what happens for the six unused key states.

**5.d.** Implement the 26-letter Caesar oracle in Qiskit. It is acceptable to use a reversible lookup/permutation implementation for this problem. Your oracle should mark valid keys satisfying

$$(19 + k) \bmod 26 = 22,$$

and should not mark unused key states.

**5.e.** Run Grover search on the 5-qubit key register. Since the physical search space has size 32, not 26, compare the iteration estimate using  $N = 32, M = 1$  with the estimate using  $N = 26, M = 1$ . Which one better matches your implemented circuit, and why?

**5.f.** Report the measurement distribution after your chosen number of Grover iterations. What probability appears on  $k = 3$ ? What probability appears on the unused states?

## Bonus Problem 6: Oracle Construction for a General Symmetric Cipher

In Problems 1–5, the encryption rule was the Caesar cipher, so we could explicitly write down the map

$$\text{Caesar}_k(p) = p + k \pmod{26}$$

or

$$\text{Caesar}_k(p) = p + k \pmod{26}.$$

Now suppose we replace Caesar encryption with an arbitrary public symmetric encryption algorithm.

Let

$$E_k(P)$$

denote the ciphertext obtained by encrypting a known plaintext  $P$  using candidate key  $k$ . You are given a known plaintext-ciphertext pair

$$(P, C),$$

where

$$C = E_{k^*}(P)$$

for some unknown secret key  $k^*$ . The goal is to use Grover search to find  $k^*$ .

Define the verification function

$$f(k) = \begin{cases} 1, & E_k(P) = C, \\ 0, & E_k(P) \neq C. \end{cases}$$

The Grover oracle should apply

$$O_f |k\rangle = (-1)^{f(k)} |k\rangle.$$

The important point is that the oracle does *not* need to know  $k^*$ . It only needs to evaluate the public test

$$E_k(P) = C$$

for each candidate key  $k$ .

**6.a.** We use three conceptual registers:

$$|k\rangle |y\rangle |a\rangle.$$

The key register  $|k\rangle$  stores the candidate key being tested. The work register  $|y\rangle$  stores the computed ciphertext  $E_k(P)$ . The ancilla register  $|a\rangle$  stores temporary scratch work used by the reversible encryption circuit.

Suppose the reversible encryption circuit  $U_E$  acts as

$$U_E : |k\rangle |0\rangle |0\rangle_a \mapsto |k\rangle |E_k(P)\rangle |0\rangle_a.$$

What is stored in each register after applying  $U_E$ ?

**6.b.** After applying  $U_E$ , the work register contains

$$y = E_k(P).$$

We now want to apply a minus sign exactly when

$$y = C.$$

Define the comparison phase operation

$$\text{PhaseEquals}_C |y\rangle = \begin{cases} -|y\rangle, & y = C, \\ |y\rangle, & y \neq C. \end{cases}$$

Why does this operation mark the correct key branch when it is applied after  $U_E$ ?

**6.c.** Suppose the ciphertext has  $b$  bits, so

$$C = C_{b-1} \cdots C_1 C_0.$$

To implement  $\text{PhaseEquals}_C$ , first transform the target value  $C$  into the all-ones string  $11 \cdots 1$ . For each bit position  $i$ , decide whether to apply an  $X$  gate to the work qubit  $y_i$ .

Which work qubits should receive  $X$  gates: those with  $C_i = 0$ , or those with  $C_i = 1$ ?

**6.d.** Once the branch  $y = C$  has been transformed into

$$|11 \cdots 1\rangle,$$

apply a multi-controlled phase gate that gives a minus sign only to

$$|11 \cdots 1\rangle.$$

Then undo the  $X$  gates from part (c).

Explain why the net effect is

$$|y\rangle \mapsto \begin{cases} -|y\rangle, & y = C, \\ |y\rangle, & y \neq C. \end{cases}$$

**6.e.** Now put the full Grover oracle together:

$$O_f = U_E^\dagger \text{PhaseEquals}_C U_E.$$

Fill in the blanks:

$$\begin{array}{l} |k\rangle |0\rangle |0\rangle_a \xrightarrow{U_E} \underline{\hspace{10em}} \\ \xrightarrow{\text{PhaseEquals}_C} \underline{\hspace{10em}} \\ \xrightarrow{U_E^\dagger} \underline{\hspace{10em}}. \end{array}$$

**6.f.** What are the final states of the work register  $|y\rangle$  and the ancilla register  $|a\rangle$  after the full oracle? Why is it important that these registers are returned to  $|0\rangle$ ?

**6.g.** Explain why this construction does not require knowing the secret key  $k^*$  in advance. What information does the circuit use instead?

**6.h.** State the final effect on the key register alone:

$$|k\rangle \mapsto \underline{\hspace{10em}}.$$

## Bonus Problem 7: Gate-Level Toy Symmetric Cipher Oracle

This problem is a fully explicit two-bit version of the previous problem. It is not secure cryptography; it is a small model for understanding the gates.

Let the key be two bits

$$k = k_1 k_0,$$

and let the plaintext be fixed to

$$P = 01.$$

Define the toy encryption rule

$$E_k(P) = P \oplus k.$$

Suppose the known ciphertext is

$$C = 10.$$

**7.a.** Solve for the correct key  $k^*$  by hand.

**7.b.** Use a two-qubit key register  $|k_1 k_0\rangle$  and a two-qubit work register  $|y_1 y_0\rangle$ . Build  $U_E$  using only  $X$  and CNOT gates:

$$|k\rangle |00\rangle \mapsto |k\rangle |P \oplus k\rangle.$$

Which  $X$  gate inserts the known plaintext bit? Which CNOTs copy the key bits into the work register by XOR?

**7.c.** Build the comparison phase for  $C = 10$ : apply  $X$  to  $y_1$  to compute  $y \mapsto y \oplus C$ ; apply  $X$  to both work qubits so the matching branch becomes 11; apply  $CZ(y_1, y_0)$ ; then undo those  $X$  gates. Check that only  $y = 10$  receives a minus sign.

**7.d.** Decompose the controlled- $Z$  using CNOT and one-qubit gates:

$$CZ = (I \otimes H) CNOT (I \otimes H),$$

where the Hadamards act on the CNOT target. Verify this identity by testing all four two-qubit basis states.

**7.e.** Put the full oracle together:

$$U_E^\dagger \text{ComparePhase}_{10} U_E.$$

Check all four basis states  $|k\rangle$ . Exactly one key branch should get a minus sign, and the work register should return to  $|00\rangle$ .

**7.f.** Run Grover search on this two-qubit key space. How many iterations should you use for  $N = 4, M = 1$ ? What success probability do you observe?

## Final Reflection

Write one concise paragraph answering this question:

In Grover key search, how can the oracle mark the unknown secret key without already knowing the secret key?

Your answer should mention that the oracle computes a public verification test, compares the computed ciphertext with the known ciphertext, applies a phase, and uncomputes temporary registers.